Hackathon Guidelines

Kombee Frontend & Backend Team Hackathon 2.0

Prepared By

Kombee

Kombee Technologies (I) Pvt. Ltd.

517-520, Homeland City,

Vesu, Surat, Gujarat – 395007

Index

1. Objective 4
2. Application Requirements (Keep It Simple) 4
3. Docker Requirement (Confidence Section) 5
4. What Is Observability? (Simple Explanation) 5
5. Observability Requirements (75% Focus) 6
6. Time Allocation (Recommendation) 8
7. Final Submission Requirements (Very Important) 9
8. Scoring System (100 Marks) 11

“If It Breaks at 2 AM, Can You Prove Why?”

⏱️ Duration: 8 Hours

🎯 Focus Split: **25% Application + Docker | 75% Observability**

Participants:

Backend Team (Java, DotNet, PHP, Python)

&

Frontend Team (Vue.js, React.js)

1. Objective

- In real production systems, writing code is only a small part of engineering.
- The real skill is:
  - Understanding how your system behaves under load
  - Detecting bottlenecks
  - Investigating failures
  - Tracing a request end-to-end
  - Proving your findings using data
  - This hackathon is designed to test your **production engineering maturity**.
- Everything required is achievable within 8 hours if:
  - You use Docker
  - You use AI to generate scaffolding
  - You limit app complexity
  - You focus heavily on observability

2. Application Requirements (Keep It Simple)

  - ⚠️ Application complexity is NOT the focus.
- ❌ Not Allowed:
  - Kombee internal admin panel generator
- ✅ Allowed:
  - Open-source boilerplate
  - AI-generated scaffolding
  - Any backend tech stack
  - Unfamiliar stack (bonus mindset)

Minimum Required Features

- Registration
- Login

  2-3 CRUD entities

- Pagination
- Filtering
- Validation
- Basic UI with variety of components:
  - Tables
  - Forms
  - Modals
  - Dropdowns
  - Alerts
  - Error messages
  - RBAC is OPTIONAL.
  - If your boilerplate works perfectly → you MUST intentionally inject anomalies later.

3. Docker Requirement (Confidence Section)

- Everything must run via Docker Compose:
  - Application
  - Database
  - Prometheus
  - Loki
  - Tempo
  - Grafana
- You are encouraged to:
  - Ask AI to generate Dockerfile
  - Ask AI to generate Docker Compose
  - Avoid manual time-consuming setup
  - ⏱️ Maximum time for App + Docker: **2 Hours**
  - After 2 hours → Freeze feature development and move to observability.

4. What Is Observability? (Simple Explanation)

- Observability means:
  - If something becomes slow or breaks, can you explain WHY using system data?
- There are 3 pillars:
- **A. Metrics (Numbers Over Time)**
- Example:
  - Requests per minute

    - Error rate
    - Response time
- Collected by:
    - Prometheus
- Visualized in:
    - Grafana
- **B. Logs (Detailed Events)**
- Example:
    - "User login failed"
    - "Database timeout"
    - "Validation error"
- Collected by:
    - Loki
- **C. Traces (Journey of a Request)**
- Example:
    - User clicks "Create Order"
- Trace shows:
    - Client → API → Controller → Service → Database → Response → Client
- Collected via:
    - OpenTelemetry
- Stored in:
    - Tempo

5. Observability Requirements (75% Focus)

- **A. Application Health Dashboard**
    - Create a Grafana dashboard similar to typical "HTTP Overview" dashboards.
- Must include:
    - Requests per minute
    - Error rate %
    - 95th percentile latency
    - Slowest endpoints
    - Active users over time
- You must answer:
    - At what load does latency increase?
    - Which endpoint becomes slow first?

- **B. Database Performance Dashboard**
- Must include:
    - Query duration graph
    - Slow queries
    - DB connection usage
    - DB error rate

- You must answer:
  - Is slowdown caused by DB or application logic?
  - Which query is the bottleneck?

- **C. Logs Dashboard**
- Using Loki, create panels showing:
  - Error logs
  - Login failures
  - Validation failures
  - Log count grouped by severity
  - Filter logs by trace ID
- You must demonstrate:
  - Pick one failed request
  - Locate it in logs
  - Use its trace ID to find its trace

- **D. Traces (Very Important Section)**
- You MUST capture one full request end-to-end:
  - Client → API → Controller → Service → DB → Same route Back to Client
- Requirements:
  - Instrument every logical layer
  - Create custom spans (logical checkpoints)
  - Show timing breakdown
- Example trace breakdown:
  - POST /orders
  - 6ms → Controller
  - 30ms → Business logic
  - 140ms → DB query
  - 8ms → Response formatting
- You must show:
  - Which span takes the most time
  - How that changes under load
  - Clear bottleneck explanation

- **E. Anomaly Injection (Mandatory)**
  - If your app works perfectly, you MUST create problems intentionally.
- Examples:
  - Add artificial delay (sleep)
  - Remove DB index
  - Create inefficient query
  - Throw random 500 errors
  - Increase payload size
- Then prove using dashboards:

  - What changed?
  - What metric increased?
  - What trace span increased?
  - What logs appeared?
  - No anomaly → No proof of observability.

- **F. Load Testing**
- You must simulate load:
- Minimum:
  - 5,000+ requests
  - Concurrent users
  - Spike test
- Bonus if using:
  - k6
- Dashboard must clearly show:
  - Latency increase
  - Error spike
  - Throughput variation

6. Time Allocation (Recommendation)

- Total: 8 Hours
- **Phase 1 - App + Docker (2 Hours)**
  - Scaffold app
  - Setup login + CRUD
- Setup DB
  - Dockerize everything
  - Confirm working
  - Stop feature development after 2 hours.

- **Phase 2 - Instrumentation (2.5 Hours)**
- Add OpenTelemetry
  - Add metrics
  - Add structured logs
  - Add trace spans
  - Verify Prometheus scraping

- **Phase 3 - Dashboard Engineering (2 Hours)**
  - Health dashboard
  - DB dashboard
  - Logs dashboard
  - Trace exploration

- **Phase 4 - Anomaly + Load Test (1 Hour)**
  - Inject problem
  - Run load test
  - Capture dashboard evidence
  - Identify bottleneck

- **Phase 5 - Recording Preparation (30 Minutes)**
  - Prepare structured explanation for video recording.

7. Final Submission Requirements (Very Important)

- Instead of live presentation, you must:

🎥 1️⃣ **Create Screen Recording Video**

- You must record a detailed walkthrough showing:
- Working application
- Grafana dashboards (explain every panel)
- Anomaly injection
- One full trace end-to-end
- Bottleneck identification
- k6 load testing execution
- Before vs After anomaly metrics
- Clear explanation of findings
- ⚠️ The video must be:
  - Very detailed
  - Clearly narrated
  - Slow and well explained
  - Professional in structure
  - Commit the video file(s) at the **root of your GitHub repository**.

📂 2️⃣ **GitHub Submission**

- Use your Kombee GitHub ID
  - Push complete project
  - Include Docker setup
  - Include dashboards JSON export
  - Include load testing script
  - Include video(s) at root
  - Submit GitHub repository link to your respective department lead
- 📅 Deadline: **14th March, 8:00 PM**

🤖 3️⃣ **AI-Based Evaluation**

- We will use AI tools to evaluate:
- Your video explanation clarity
- Your Dockerfile structure
- Your Docker Compose architecture
- Observability depth
- Trace completeness
- Dashboard quality
- Since AI will evaluate:
    - Make your video extremely detailed
    - Explain reasoning clearly
    - Avoid skipping steps
    - Show real evidence
    - Superficial explanations will score lower.

8. Scoring System (100 Marks)

- **Application (10 Marks)**
    - Working login + CRUD (5)
    - Clean structure (5)

- **Docker Setup (10 Marks)**
    - Proper Dockerfile (5)
    - Full stack orchestration (5)
- **Metrics (15 Marks)**
    - Request metrics (5)
    - Latency percentiles (5)
    - Business metrics (5)

- **Logs (15 Marks)**
    - Structured logging (5)
    - Filtering & correlation (5)
    - Error analysis (5)

- **Traces (20 Marks)**
    - End-to-end request captured (5)
    - Custom spans (5)
    - Timing breakdown clarity (5)
    - Bottleneck identification (5)

- **Dashboards Quality (15 Marks)**
    - Clean visualization (5)
    - Insightful interpretation (5)
    - Logical layout (5)

- **Anomaly Injection & Analysis (10 Marks)**
  - Meaningful anomaly (5)
  - Data-backed explanation (5)

- **Load Testing (5 Marks)**
  - Load simulation (3)
  - k6 bonus (2)

- **Final Thought**
  - This hackathon is not about who writes the most code.
  - It is about who can confidently answer:
  - If this system slows down in production…
  - Can you prove exactly where and why?
  - That is real backend engineering.